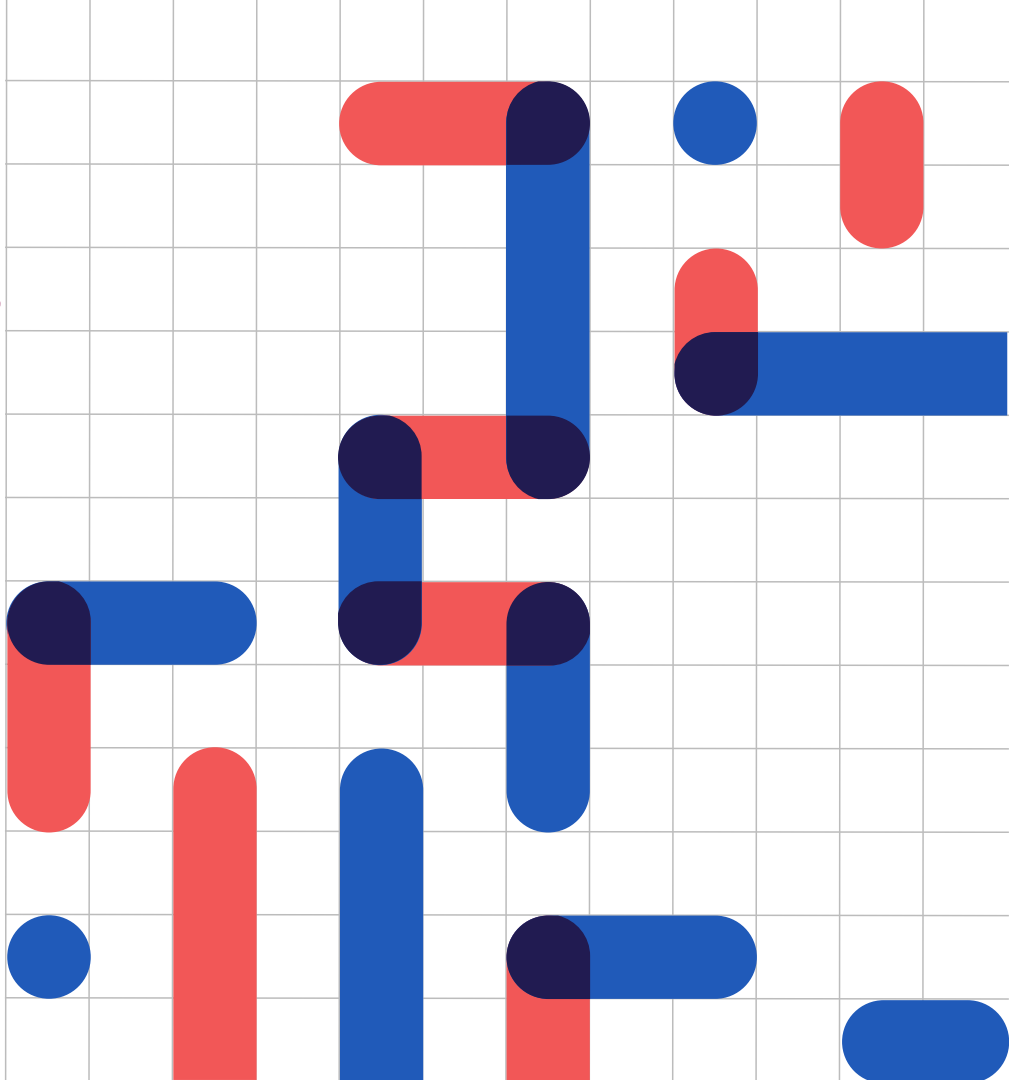
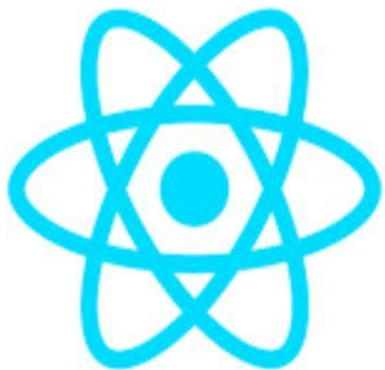


React JS...



What is React?

Definition:

JavaScript library for building user interfaces with reusable components

Created by:

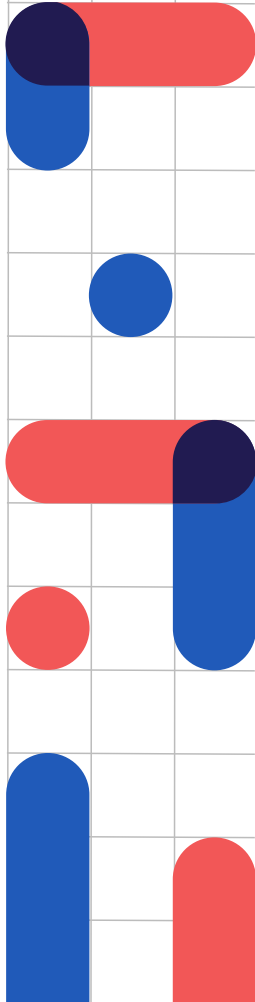
Facebook (2011) • Open-sourced (2013)

Core Purpose:

Make building dynamic UIs easier, faster, and more efficient

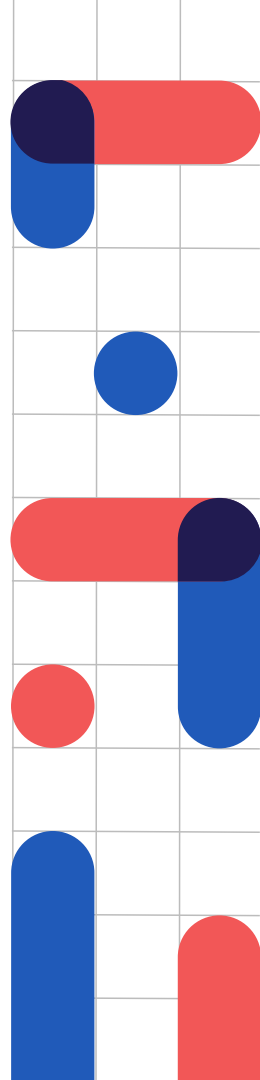
Status in 2026:

Most popular UI library • Industry standard



Key Feature

- ❑ Component-based architecture
- ❑ JSX syntax
- ❑ Virtual DOM
- ❑ Reusable UI components
- ❑ Strong ecosystem



Components

ReactJS follows a component-based architecture, where UIs are composed of reusable building blocks called components.

React supports two ways of declaring a component

❖ Functional Components

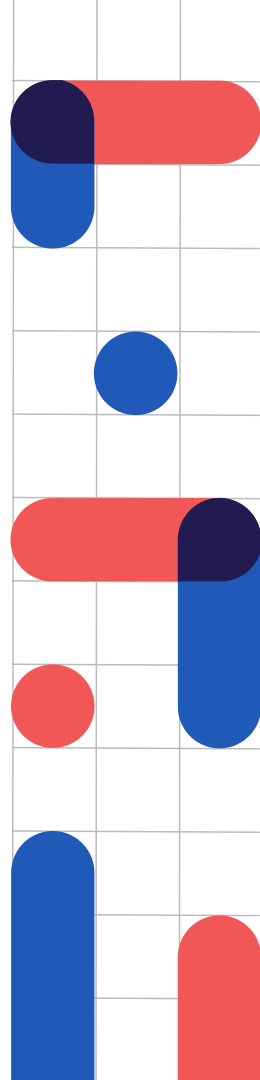
- ❖ Simple JavaScript functions
- ❖ Most commonly used today
- ❖ Can use Hooks

```
function Hello()  
{  
  return <h1>Hello React</h1>;  
}
```

❖ Class Components

- ❖ ES6 classes
- ❖ Older approach

```
class Hello extends React.Component {  
  render() {  
    return <h1>Hello React</h1>;  
  }  
}
```



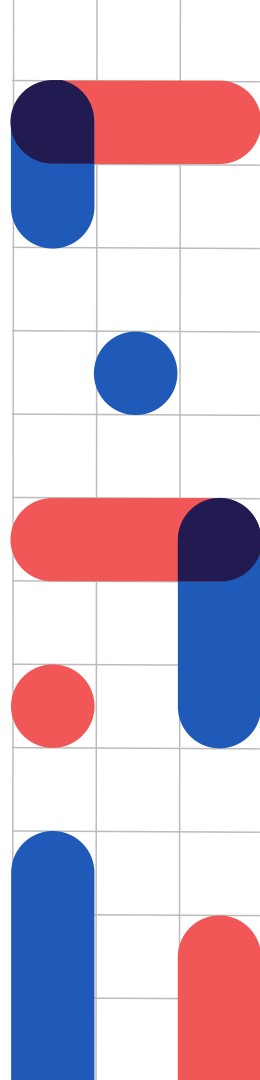
JSX

Allows writing HTML inside JavaScript

Makes UI code readable

Must return one parent element

```
const element = <h1>Hello World</h1>;
```



Virtual DOM

What is it?

JavaScript representation of actual DOM (in-memory abstraction)

How it works:

1. Component renders Virtual DOM created  State changes  New

Reconciliation:

React compares old vs new Virtual DOM (Diffing)

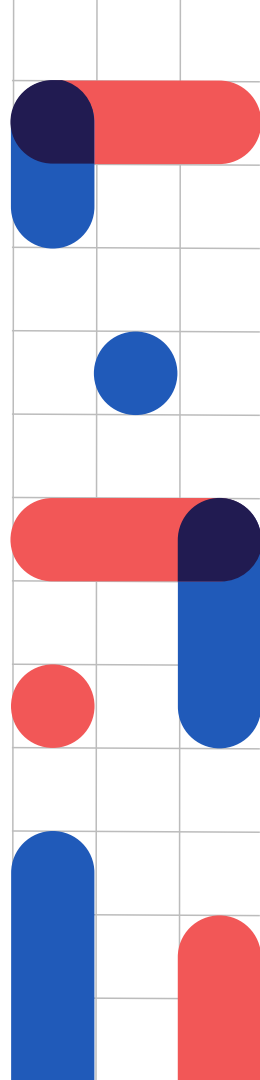
Efficiency:

Only differences updated in real DOM

Benefit:

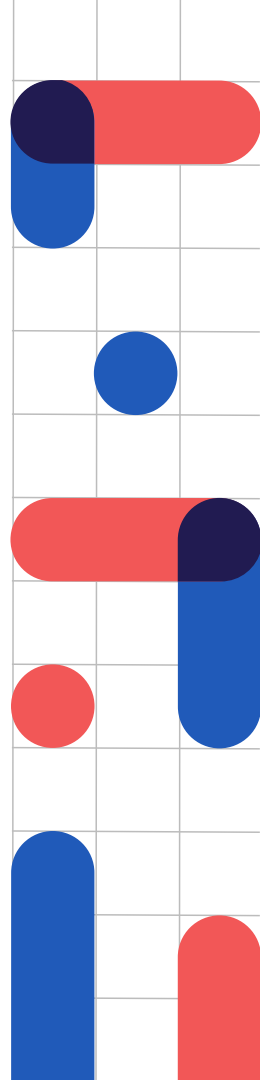
Better performance

Smoother user experience



React DOM & Rendering

- ❑ React DOM connects React components to the browser's real DOM
- ❑ React creates a Virtual DOM (lightweight copy in memory)
- ❑ On changes in state or props, React:
 - ❑ Updates Virtual DOM
 - ❑ Compares old vs new (diffing)
 - ❑ Updates only changed elements in Real DOM



Single Page Applications

What?

Web app that loads ONE HTML page and dynamically updates content

No Full Reloads:

Navigation happens client-side without page refresh

Smooth I.JX:

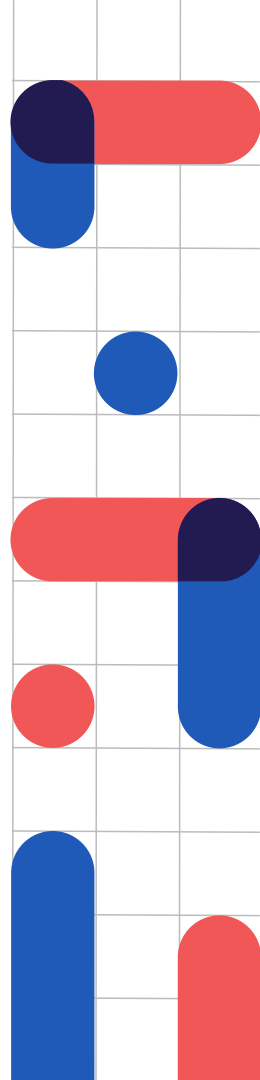
Desktop-like experience, instant interactions

Data Transfer:

Only JSON data fetched, not entire HTML pages

React Perfect for SPAs:

Components, routing, state management all built-in



React Hooks: useState & useEffect

Hooks add features to functional components

useState

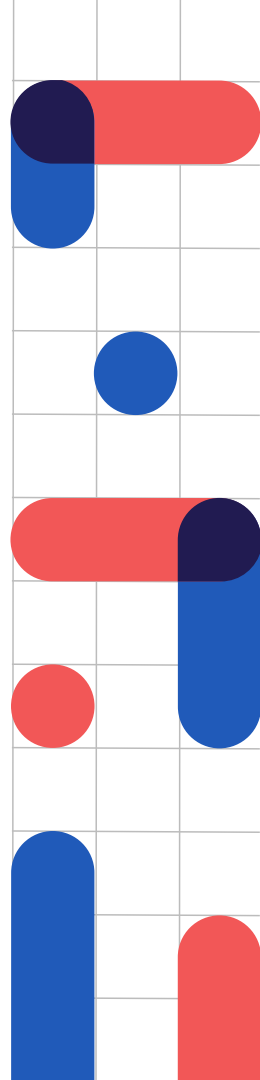
```
const [count, setCount] = useState(0);
```

- Manages component **state**
- Updating state triggers **re-render**

useEffect

- Handles **side effects** (API calls, timers, DOM updates)
- Runs after render

```
useEffect(() => {  
  // side effect  
}, []);
```



React Hooks: useContext

Purpose:

Access context values without prop drilling

Problem it solves:

Avoid passing props through many intermediate components

When to use:

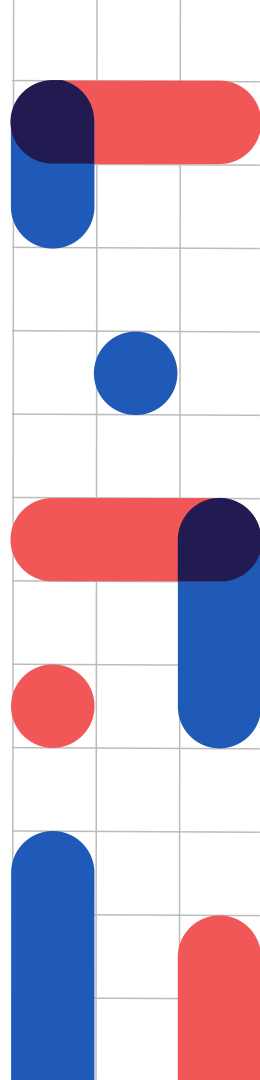
Theme settings, user auth, global UI state

When NOT to use:

Frequent updates (performance issue)

Modern Alternative:

TanStack Query for server state



How React Works: Reconciliation

Reconciliation:

Process of comparing old and new Virtual DOM

The Algorithm:

Render + Compare + Diff + Patch + Commit

Key Concept:

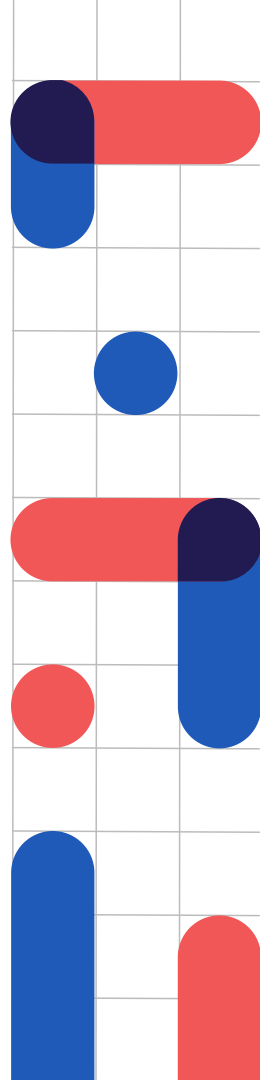
Using proper keys (not array index) is critical

Fiber Architecture:

React's internal engine for incremental rendering

Result:

Smooth 60fps UI even during heavy computation

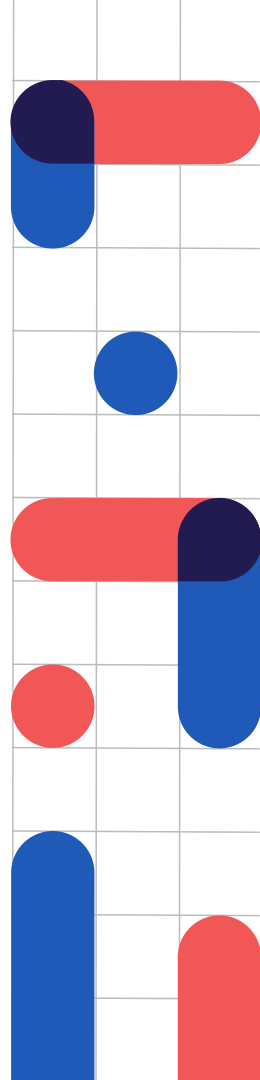


REST API Integration in React

- ❑ REST API allows React apps to communicate with a backend server
- ❑ API calls are usually placed inside `useEffect`
- ❑ Fetched data is stored using `useState`

Common tools:

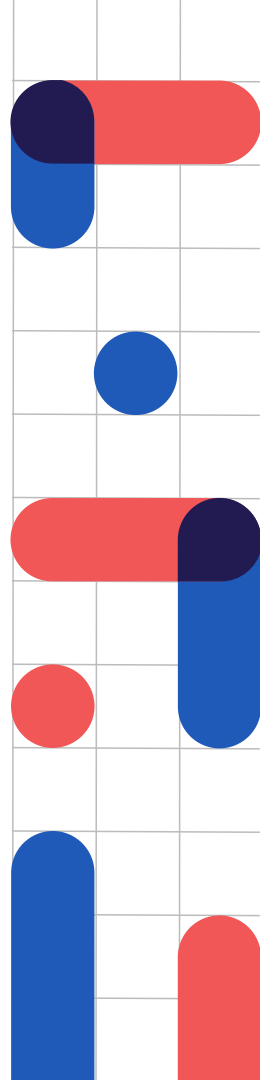
- ❑ `fetch()`
- ❑ `Axios`



How React Connects to Server (Frontend-Backend flow)

React UI → API Request → Backend → Database → JSON Response

- ❑ User interacts with React UI (click, submit, load page)
- ❑ React triggers an API request using fetch or axios
- ❑ Request is sent to the Backend Server via HTTP (GET, POST, PUT, DELETE)
- ❑ Backend processes request
- ❑ Server sends back a response (JSON data)
- ❑ React receives data and stores it in state
- ❑ UI re-renders with updated data



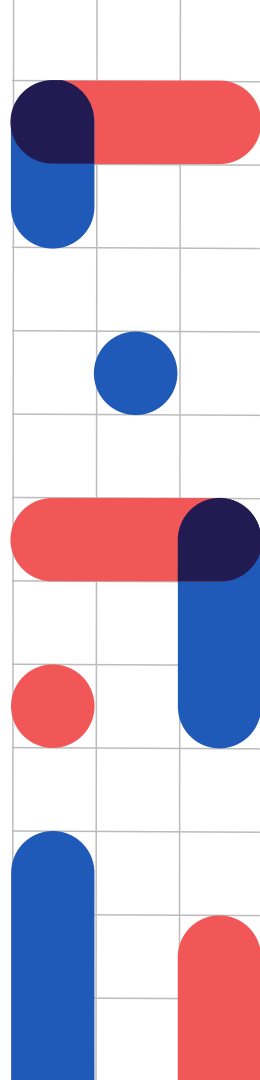
React: Advantages & Ecosystem

Advantages

- ❑ Component-based architecture
- ❑ Reusable and maintainable code
- ❑ Fast rendering using Virtual DOM
- ❑ Strong community support
- ❑ Easy integration with other libraries

Ecosystem

- ❑ React Router → Navigation
- ❑ Redux / Context API → State management
- ❑ Axios / Fetch → API calls
- ❑ Next.js → SSR & performance
- ❑ Material UI / Bootstrap → UI components



THANKYOU..

Why Use React in 2026?

- ❑ Industry standard with largest ecosystem
- ❑ Most job opportunities for developers
- ❑ Server Components for improved performance
- ❑ Excellent community support & documentation
- ❑ Works with modern tools (Next.js, TypeScript, Vite)
- ❑ Future-proof with continued Meta investment

